

Maré: Especificação e Implementação dos Analisadores Léxico e Sintático de uma Linguagem Imperativa Didática em Português

Gustavo Teixeira Bittencourt de Oliveira, Christophe Abelem Pinho,
Edgar Klewert Da Costa Araujo, Elissandra Bernadett Abdon Nascimento, Adler Augustus de Castro Mota
Curso de Engenharia da Computação – Turma EC8MA
Centro Universitário do Estado do Pará (CESUPA)
Belém, PA, Brasil

Resumo—Este artigo descreve a primeira etapa da construção de um compilador para a Maré, uma linguagem de programação imperativa, estaticamente tipada e com palavras-chave em português, projetada pela equipe. São apresentados a especificação léxica da linguagem, com suas classes de lexemas, tokens e expressões regulares, e uma gramática livre de contexto em notação EBNF que cobre declarações de variáveis e constantes, o bloco principal, condicionais encadeados, três formas de laço, funções com parâmetros tipados e expressões aritméticas, relacionais e lógicas. O analisador léxico foi implementado em Python, sem geradores automáticos, a partir de uma expressão regular mestre com grupos nomeados. O analisador sintático é descendente recursivo, constrói uma árvore sintática abstrata e recupera-se de erros em modo pânico. Um programa de demonstração com 60 linhas é convertido em uma cadeia de 288 tokens e aceito pelo analisador sintático, e 28 testes automatizados validam precedência, associatividade e o diagnóstico de erros.

Index Terms—compiladores, análise léxica, análise sintática, expressões regulares, gramática livre de contexto, parser descendente recursivo.

I. INTRODUÇÃO

Um compilador traduz um programa escrito em uma linguagem-fonte para uma representação equivalente em outra linguagem, e o faz em fases bem delimitadas [1]. As duas primeiras formam a *vanguarda* (*front end*) do processo: a análise léxica agrupa os caracteres do código em unidades com significado, os *tokens*, e a análise sintática verifica se a sequência desses tokens respeita a estrutura da linguagem, produzindo uma árvore que servirá às fases seguintes [2].

O objetivo deste trabalho é construir essas duas fases para uma linguagem nova, a **Maré**, que servirá de protótipo ao longo do semestre. A escolha por uma linguagem própria, em vez de um subconjunto de uma linguagem existente, permitiu controlar cada decisão de projeto e justificá-la do ponto de vista da teoria de linguagens formais. O nome faz referência ao fenômeno das marés, característico da região amazônica, e também sugere o fluxo de tokens que “sobe” do código-fonte até a árvore sintática.

As contribuições deste artigo são: (i) a especificação léxica e sintática da Maré; (ii) um analisador léxico baseado em expressões regulares com detecção de erros; (iii) um analisador

sintático descendente recursivo com construção de árvore sintática abstrata (AST) e recuperação de erros; e (iv) a validação da implementação por meio de programas de exemplo e testes automatizados.

O restante do texto está organizado da seguinte forma: a Seção II revisa os conceitos utilizados; a Seção III apresenta a linguagem; as Seções IV e V detalham os analisadores; a Seção VI discute os resultados; e a Seção VII conclui o trabalho.

II. FUNDAMENTAÇÃO TEÓRICA

A. Lexemas, tokens e padrões

Um *lexema* é uma sequência concreta de caracteres do código-fonte, como `notal` ou `>=`. Um *token* é o par formado por um nome abstrato, que indica a classe do lexema, e um atributo opcional, geralmente o próprio lexema [1]. O *padrão* descreve o conjunto de lexemas que pertencem a uma classe e é normalmente expresso por uma expressão regular. As linguagens regulares são reconhecidas por autômatos finitos, o que torna a análise léxica executável em tempo linear no tamanho da entrada [3].

Quando mais de um padrão casa com o mesmo ponto da entrada, duas regras resolvem a ambiguidade: a do *casamento mais longo*, que prefere o maior lexema possível (`<=` em vez de `<`), e a da *prioridade*, que, entre casamentos do mesmo tamanho, escolhe o padrão listado primeiro [2].

B. Gramáticas livres de contexto

A estrutura sintática de uma linguagem de programação é descrita por uma gramática livre de contexto (GLC) $G = (N, \Sigma, P, S)$, em que N é o conjunto de não-terminais, Σ o de terminais (aqui, os tokens), P o de produções e S o símbolo inicial [3]. A notação EBNF estende a BNF com operadores de repetição $\{ \}$ e de opcionalidade $[\]$, o que torna a gramática mais compacta [6].

C. Análise descendente recursiva

Um analisador descendente recursivo associa a cada não-terminal uma função que reconhece as suas produções, consumindo tokens da esquerda para a direita [4]. Para que a escolha

da produção seja determinística com k tokens de antecipação (*lookahead*), a gramática deve ser $LL(k)$: sem recursão à esquerda e com conjuntos FIRST disjuntos entre alternativas [5]. Essa estratégia dispensa ferramentas geradoras, produz código legível e facilita mensagens de erro específicas, razões pelas quais é adotada por compiladores de produção e recomendada em contextos didáticos [7].

III. A LINGUAGEM MARÉ

A Maré é imperativa, estruturada em blocos delimitados por chaves e estaticamente tipada, com quatro tipos primitivos: inteiro, real, logico e texto. Três decisões de projeto guiaram a sintaxe:

- **Tipo à direita do nome:** seja x : inteiro = 1;. A palavra inicial (seja ou fixo) identifica a declaração com um único token de antecipação.
- **Operadores lógicos por extenso** (e, ou, nao), o que aproxima as condições da leitura em português.
- **Blocos sempre entre chaves**, inclusive em comandos de uma só linha, o que elimina a ambiguidade do *senão pendurado*.

A Listagem 1 mostra um trecho do programa de demonstração. Ele contém os recursos mínimos exigidos (variáveis, bloco principal, condicionais, equações matemáticas e regras lógicas) e também funções e laços.

Listagem 1. Trecho do programa de demonstração completo.mare.

```
1  fixo MEDIA_MINIMA: real = 7.0;
2
3  funcao media(p1: real, p2: real,
4             peso: inteiro) -> real {
5      retorne (p1 + p2 * peso) / (1 + peso);
6  }
7
8  principal {
9      seja nota1: real = 6.5;
10     seja nota2: real;
11     seja faltas: inteiro = 3;
12     leia(nota2);
13     seja m: real = media(nota1, nota2, 2);
14
15     # regra logica composta
16     se (m >= MEDIA_MINIMA e faltas <= 10
17        ou nota2 == 10.0) {
18         mostre("aprovada com media ", m);
19     } senao se (m >= 5.0 e nao (faltas > 10)) {
20         mostre("em recuperacao");
21     } senao {
22         mostre("reprovada");
23     }
24     para k de 1 ate 10 passo 2 {
25         mostre(k, " ao quadrado = ", k ^ 2);
26     }
27     repita {
28         faltas = faltas - 1;
29     } ate (faltas <= 0);
30 }
```

A Tabela I resume os comandos disponíveis.

IV. ANALISADOR LÉXICO

A. Classes de lexemas e tokens

A Maré possui 50 tipos de token, organizados em dez classes de lexemas (Tabela II). Os operadores lógicos, em-

Tabela I
RECURSOS DA LINGUAGEM MARÉ

Recurso	Sintaxe
Variável	seja x : tipo [= expr];
Constante	fixo X : tipo = expr;
Bloco principal	principal { ... }
Condicional	se (c) {} [senao {}]
Senão-se	senao se (c) {}
Laço pré-teste	enquanto (c) { ... }
Laço contado	para i de a ate b [passo p] {}
Laço pós-teste	repita { ... } ate (c);
Função	funcao f(a: tipo) [-> tipo] {}
Retorno	retorne [expr];
Entrada/Saída	leia(x); mostre(e1, e2);
Comentários	# linha /* bloco */

Tabela II
CLASSES DE LEXEMAS E TOKENS DA MARÉ

Classe	Tokens	Lexemas
Palavra reservada	PRINCIPAL, FUNCAO, SEJA, FIXO, SE, SENAO, ENQUANTO, PARA, DE, ATE, PASSO, REPITA, RETORNE, MOSTRE, LEIA	principal funcao seja fixo se senao enquanto para de ate passo repita retorne mostre leia
Tipo primitivo	T_INTEIRO, T_REAL, T_LOGICO, T_TEXTO	inteiro real logico texto
Identificador	ID	nota1, eh_par
Líteral	NUM_INT, NUM_REAL, TEXTO, VERDADEIRO, FALSO	42, 7.0, "Ana", verdadeiro
Op. aritmético	MAIS, MENOS, MULT, DIV, MOD, POT	+ - * / % ^
Op. relacional	IGUAL, DIFERENTE, MENOR, MENOR_IGUAL, MAIOR, MAIOR_IGUAL	== != < <= > >=
Op. lógico	E, OU, NAO	e ou nao
Atribuição	TRIB	=
Delimitador	ABRE_PAR, FECHA_PAR, ABRE_CHAVE, FECHA_CHAVE, PONTO_VIRGULA, VIRGULA, DOIS_PONTOS, SETA	() { } ; , : ->
Fim	EOF	(fim do arquivo)

bora sejam palavras, formam uma classe própria, porque na gramática ocupam a posição de operadores e não de comandos.

B. Expressões regulares

Cada classe é definida por uma expressão regular (Tabela III). Espaços e comentários são reconhecidos e descartados. Algumas regras não produzem tokens: existem para capturar entradas inválidas e emitir mensagens de erro precisas.

C. Justificativa da expressão regular dos identificadores

O identificador é definido por

$$ID = (letra \mid _) (letra \mid digito \mid _)*$$

com $letra = [A-Za-z]$ e $digito = [0-9]$, o que corresponde à regex $[A-Za-z_][A-Za-z0-9_]*$. Cada parte da definição tem um motivo:

- **Primeiro caractere sem dígito.** Um lexema que começa por dígito é sempre número. Com isso, a primeira posição

Tabela III
EXPRESSÕES REGULARES DO SCANNER, EM ORDEM DE PRIORIDADE

# Regra	Expressão regular
1 ESPACO	[\t\r]+
2 NOVA_LINHA	\n
3 COMENTARIO_BLOCO	\#*\[\s\S]*?*\#
4 COMENTARIO_ABERTO†	\#*
5 COMENTARIO_LINHA	\#[^\n]*
6 NUM_INVALIDO†	\d+(?:\.\d+)?[A-Za-z_]\w*
7 NUM_REAL	\d+\.\d+
8 NUM_INT	\d+
9 TEXTO	"(?:\\. [^\\"\\n])*"
10 TEXTO_ABERTO†	"(?:\\. [^\\"\\n])"
11 ID	[A-Za-z_][A-Za-z0-9_]*
12 SIMBOLO	-> = != < = <=>+ - * /%^(){};,:
13 DESCONHECIDO†	.

† Regra de erro: não gera token, registra um erro léxico.

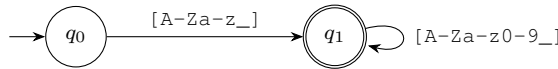


Figura 1. AFD que reconhece a classe ID.

decide entre as classes NUM e ID, e 12abc pode ser tratado como erro sem ambiguidade (regra 6).

- **Sublinhado permitido.** Permite nomes compostos legíveis (eh_par) e a convenção de constantes em maiúsculas (MEDIA_MINIMA).
- **Somente ASCII.** Letras acentuadas ficam fora do alfabeto de identificadores, e preço gera erro com a dica “identificadores não aceitam acentos”. Assim o conjunto de caracteres válidos é o mesmo em qualquer sistema operacional ou codificação do arquivo, e as palavras reservadas não têm grafias duplicadas (senao e senão).
- **Fecho de Kleene no restante.** Não há limite de tamanho, e o casamento mais longo garante que nota12 seja um único token.

A Fig. 1 mostra o autômato finito determinístico equivalente. O estado q_1 é de aceitação, e qualquer caractere fora de $[A-Za-z0-9_]$ encerra o lexema.

D. Justificativa da tabela de palavras reservadas

As 24 palavras reservadas, os tipos e os operadores lógicos também pertencem à linguagem de ID. Em vez de escrever uma expressão regular para cada uma, o scanner reconhece o lexema como identificador e depois consulta uma *tabela de palavras reservadas*, implementada como dicionário. Se o lexema estiver na tabela, o token é reclassificado. Essa escolha tem três vantagens: (i) o casamento mais longo é preservado sem regras extras, e por isso seja é identificador enquanto seja é reservada, um erro comum quando se escreve seja como regex antes de ID; (ii) a consulta custa $O(1)$ em média; e (iii) incluir uma palavra nova na linguagem exige apenas uma linha na tabela. Os símbolos seguem a mesma ideia: uma regex única os reconhece, e a tabela SIMBOLOS associa cada lexema ao seu token.

E. Implementação

O scanner foi escrito em Python sem bibliotecas de terceiros. As 13 regras da Tabela III são concatenadas em uma única *regex mestre*, na forma $(?P<ESPACO>...) | (?P<NOVA_LINHA>...) | \dots$, e aplicada com `re.finditer` sobre todo o código [8]. Como o motor de expressões regulares do Python testa as alternativas da esquerda para a direita, a ordem das regras implementa a regra da prioridade. As ambiguidades da linguagem são resolvidas por essa ordem:

- NUM_INVALIDO antes de NUM_INT: 12abc é reportado como erro, e não aceito como 12 seguido de abc;
- NUM_REAL antes de NUM_INT: 3.14 é um só token;
- operadores de dois caracteres antes dos de um dentro de SIMBOLO: <= não se divide em < e =;
- comentário de bloco (#*) antes do de linha (#).

O nome do grupo que casou (`match.lastgroup`) indica a classe do lexema. Cada token guarda tipo, lexema, linha e coluna. A linha é incrementada a cada NOVA_LINHA e também pelas quebras dentro de comentários de bloco. Os erros léxicos são acumulados em uma lista, e a varredura continua, de modo que todos são apresentados ao usuário em uma única execução.

F. Cadeia de tokens

A Listagem 2 mostra um programa mínimo e a Listagem 3 a cadeia de tokens produzida para ele, agrupada por linha. Tokens com atributo (ID, números e textos) exibem o lexema.

Listagem 2. Programa mini.mare.

```
1 principal {
2     seja x: inteiro = 2 + 3 * 4;
3     se (x > 10 e nao falso) {
4         mostre(x);
5     }
6 }
```

Listagem 3. Cadeia de tokens gerada para mini.mare.

```
1 <PRINCIPAL> <ABRE_CHAVE>
2 <SEJA> <ID,x> <DOIS_PONTOS> <T_INTEIRO> <ATRIB>
  <NUM_INT,2> <MAIS> <NUM_INT,3> <MULT> <NUM_INT,4>
  <PONTO_VIRGULA>
3 <SE> <ABRE_PAR> <ID,x> <MAIOR> <NUM_INT,10> <E>
  <NAO> <FALSO> <FECHA_PAR> <ABRE_CHAVE>
4 <MOSTRE> <ABRE_PAR> <ID,x> <FECHA_PAR> <PONTO_VIRGULA>
5 <FECHA_CHAVE>
6 <FECHA_CHAVE>
7 <EOF>
```

Além da cadeia, a ferramenta imprime a *tabela de lexemas e tokens*, que lista cada lexema distinto com seu token, sua classe e o número de ocorrências. Para o programa de demonstração completo (60 linhas), foram reconhecidos 288 tokens: 111 delimitadores, 52 identificadores, 34 literais, 33 palavras reservadas, 16 operadores aritméticos, 15 tipos, 11 atribuições, 9 operadores relacionais, 6 operadores lógicos e o EOF.

Tabela IV
PRECEDÊNCIA (CRESCENTE) E ASSOCIATIVIDADE

Nível	Regra	Operadores	Assoc.
1	ou	ou	esquerda
2	e	e	esquerda
3	nao	nao	prefixo
4	relacional	== != < <= > >=	nenhuma
5	soma	+ -	esquerda
6	termo	* / %	esquerda
7	unario	- (unário)	prefixo
8	potencia	^	direita

V. ANALISADOR SINTÁTICO

A. Gramática

A Listagem 4 apresenta a gramática da Maré em EBNF. Os terminais aparecem entre aspas ou em maiúsculas (tokens produzidos pelo scanner), e o símbolo inicial é `programa`. As regras se dividem em três grupos: estrutura (programa, funções e blocos), comandos e expressões.

Listagem 4. Gramática livre de contexto da Maré (EBNF).

```

programa = {funcao | declaracao} "principal" bloco EOF
funcao  = "funcao" ID "(" [parametros] ")"
        ["->" tipo] bloco
parametros = parametro {"," parametro}
parametro = ID ":" tipo
tipo      = "inteiro" | "real" | "logico" | "texto"
bloco     = "{" {comando} "}"

comando   = declaracao | atribuicao | chamada ";"
        | se | enquanto | para | repita
        | retorne | mostre | leia
declaracao = "seja" ID ":" tipo ["=" expr] ";"
        | "fixo" ID ":" tipo "=" expr ";"
atribuicao  = ID "=" expr ";"
se         = "se" "(" expr ")" bloco
        ["senao" (se | bloco)]
enquanto  = "enquanto" "(" expr ")" bloco
para      = "para" ID "de" expr "ate" expr
        ["passo" expr] bloco
repita    = "repita" bloco "ate" "(" expr ")" ";"
retorne   = "retorne" [expr] ";"
mostre    = "mostre" "(" argumentos ")" ";"
leia      = "leia" "(" ID ")" ";"

expr      = ou
ou        = e {"ou" e}
e         = nao {"e" nao}
nao       = "nao" nao | relacional
relacional = soma [{"="|"!="|"<"|<="|>"|>="} soma]
soma      = termo [{"+"|"-" } termo]
termo     = unario [{"*"|" /"|"%" } unario]
unario    = "-" unario | potencia
potencia  = primario [{"^" } unario]
primario  = NUM_INT | NUM_REAL | TEXTO | "verdadeiro"
        | "falso" | ID | chamada | "(" expr ")"
chamada   = ID "(" [argumentos] ")"
argumentos = expr {"," expr}

```

B. Precedência e associatividade

A precedência dos operadores não é tratada por tabelas externas. Ela está embutida na hierarquia das regras de expressão: quanto mais profunda a regra, maior a precedência do operador que ela reconhece (Tabela IV). Na expressão $2 + 3 * 4$, por exemplo, a regra soma só consome o $+$ depois que termo já reconheceu $3 * 4$ por inteiro, e a multiplicação fica mais funda na árvore.

A associatividade à esquerda de $+$, $-$, $*$ e $/$ vem da forma iterativa das regras (soma = termo { $("+" | "-")$ termo }): a cada volta do laço, a árvore acumulada torna-se o filho esquerdo do novo nó. Assim, $10 - 3 - 2$ resulta em $(10 - 3) - 2$. A potência é associativa à direita, como na matemática: potencia chama unario, que pode voltar a potencia, e por isso $2 ^ 3 ^ 2$ vale 2^9 . Os operadores relacionais foram definidos como não associativos. Uma cadeia como $a < b < c$ gera um erro sintático que sugere o uso de `e`, pois comparar um valor lógico com um número não teria sentido.

C. Propriedade LL e estratégia de análise

A gramática não tem recursão à esquerda, e quase todas as alternativas de comando começam por uma palavra reservada distinta. Por isso, um token de antecipação basta para escolher a produção. A única exceção são os comandos iniciados por `ID`, que podem ser uma atribuição ($x = \dots$) ou uma chamada ($f(\dots)$). Nesse ponto o parser examina um segundo token (`=` ou `()`), o que torna essa alternativa LL(2). O mesmo ocorre em `primario`, para distinguir uma variável de uma chamada de função.

O parser é descendente recursivo: cada não-terminal corresponde a um método da classe `Parser`, e a *docstring* de cada método reproduz a produção que ele implementa. A Listagem 5 mostra o método da regra soma. Repetições EBNF viram laços `while`, opcionais viram `if`, e alternativas viram testes sobre o token atual.

Listagem 5. Método da regra soma (simplificado).

```

def _soma(self):
    # soma -> termo { ("+"|"-") termo }
    no = self._termo()
    while op := self._aceitar(T.MAIS, T.MENOS):
        no = Binaria(op.lexema, no, self._termo())
    return no

```

D. Integração com o analisador léxico

Os dois analisadores se comunicam pela cadeia de tokens. O scanner produz a lista completa, terminada em EOF, e o parser a percorre com um índice. Quatro primitivas concentram o acesso: `atual` (token sob análise), `espiar` (antecipação de k tokens), `aceitar` (consome o token se ele for do tipo esperado) e `esperar` (consome o token obrigatório ou lança um erro sintático). Como cada token carrega linha e coluna, os erros sintáticos são localizados com precisão no código-fonte. Quando há erros léxicos, a análise sintática não é executada.

E. Árvore sintática

Durante o reconhecimento, cada método constrói e devolve um nó da árvore sintática abstrata. Os nós são classes de dados (`Programa`, `Funcao`, `Declaracao`, `Se`, `Enquanto`, `Para`, `Repita`, `Binaria`, `Unaria`, `Literal`, `Variavel`, `Chamada` etc.) que expõem os métodos `rotulo()` e `filhos()`. Isso permite que uma única rotina desenhe a árvore no terminal e a exporte para Graphviz ou para $\text{\LaTeX}$. A Fig. 2 mostra a árvore do programa da

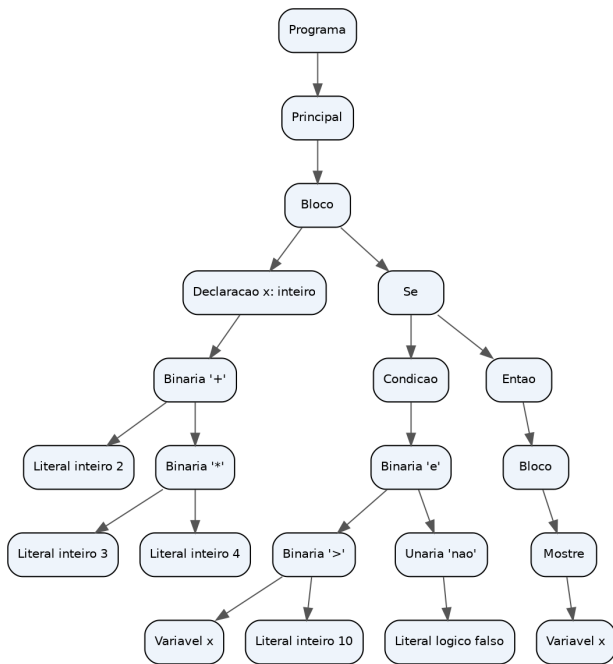


Figura 2. Árvore sintática gerada pelo analisador para `mini.mare` (exportada em formato DOT e desenhada com Graphviz).

Listagem 2. Nela, a precedência de `*` sobre `+` e de `>` sobre `e` fica visível na profundidade dos nós.

A AST difere da árvore de derivação completa porque omite os nós intermediários sem conteúdo. Uma expressão como `x`, por exemplo, não aparece como a cadeia `expr` → `ou` → `...` → `primario`, e sim como um único nó `Variavel`. A AST também dispensa tokens de pontuação, cuja função se esgota na análise. Essa forma compacta é a que será percorrida pelas fases seguintes.

F. Derivação

Para relacionar a gramática com a árvore, a derivação mais à esquerda da expressão `2 + 3 * 4` é mostrada abaixo, em que $\Rightarrow^*$ resume as cadeias de regras unitárias (`relacional` $\Rightarrow$ `soma`, `unario` $\Rightarrow$ `potencia` etc.):

$$\begin{aligned} \text{expr} &\Rightarrow^* \text{soma} \Rightarrow \text{termo} + \text{termo} \\ &\Rightarrow^* 2 + \text{termo} \Rightarrow 2 + \text{unario} * \text{unario} \\ &\Rightarrow^* 2 + 3 * \text{unario} \Rightarrow^* 2 + 3 * 4 \end{aligned}$$

Como o `*` só pode ser produzido dentro de `termo`, nenhuma derivação coloca `2 + 3` como operando da multiplicação. A gramática, portanto, não é ambígua quanto à precedência.

G. Tratamento de erros

Ao encontrar um token inesperado, o parser gera uma mensagem que informa o que era esperado, em que contexto e o que foi encontrado, e imprime a linha do código com um marcador sob a coluna. O ponto e vírgula esquecido recebe um tratamento especial: se o token seguinte estiver em outra linha, o erro é apontado no fim da linha anterior, onde o programador de fato errou.

A recuperação usa o *modo pânico* [1]. O erro é registrado, e o parser descarta tokens até um ponto de sincronização, que pode ser um `;`, um `}` ou o início de outro comando. Um bloco `{ ... }` encontrado nesse percurso é descartado por inteiro, com contagem de chaves, para que o seu `}` não feche o bloco externo por engano. Com isso, vários erros são relatados em uma única execução, sem cascatas de mensagens falsas.

VI. RESULTADOS

A implementação tem cerca de 1.150 linhas de Python, distribuídas em seis módulos (`tokens`, `lexer`, `parser`, `arvore`, `erros` e `exibicao`), e uma interface de linha de comando que exhibe, sob demanda, as expressões regulares, a cadeia de tokens, a tabela de lexemas e a árvore sintática. O programa de demonstração completo, de 60 linhas, que usa todos os recursos da Tabela I, foi aceito, e sua árvore reflete a precedência esperada em todas as expressões.

Para verificar a detecção de erros, foram criados dois arquivos com falhas intencionais. No léxico, foram identificados os cinco erros inseridos: um caractere acentuado em identificador (`preço`), um número malformado (`12abc`), um real incompleto (`3.`), um texto sem aspas de fechamento e um símbolo fora do alfabeto (`@`). No sintático, a Listagem 6 reproduz os seis erros relatados em uma única execução, cada um na posição correta.

Listagem 6. Saída para `erro_sintatico.mare` (resumida).

```
[2:16] esperado ':' apos o parametro 'n',
      mas foi encontrado 'inteiro'
[7:24] esperado ';' ao final da declaracao
      (faltou ';' no fim da linha)
[8:29] esperado ')' para fechar a expressao entre
      parenteses, mas foi encontrado ';'
[9:8]  esperado '(' apos 'se', mas foi encontrado 'a'
[12:5] 'senao' sem um 'se' correspondente
[15:21] comparacoes nao podem ser encadeadas;
      use 'e' (ex.: a < b e b < c)
```

A. Rastreamento de um caso rejeitado

Para mostrar o percurso completo de código, tokens e reconhecimento, a ferramenta tem a opção `--rast`, que imprime cada regra aplicada ($\blacktriangleright$), cada token consumido ($\checkmark$) e cada erro ou descarte ($\times$). O arquivo `rejeitado.mare` é o mesmo `mini.mare` da Listagem 2, com um único erro: falta o `)` que fecha a condição, na linha 4 (`se (x > 10 e nao falso {`). A cadeia de tokens dessa linha é

```
<SE> <ABRE_PAR> <ID,x> <MAIOR> <NUM_INT,10> <E> <NAO>
<FALSO> <ABRE_CHAVE>
```

e a Listagem 7 traz o trecho final do rastreamento.

Listagem 7. Trecho do rastreamento de `rejeitado.mare`.

```
> se      [proximo: <SE,'se'>]
v consome <SE,'se'>           (4:5)
v consome <ABRE_PAR,'('>      (4:8)
> expr    [proximo: <ID,'x'>]
...      (x > 10 e nao falso reconhecido)
v consome <FALSO,'falso'>     (4:22)
x ERRO [4:28] esperado ')' para fechar a
      condicao do 'se', mas foi encontrado '{'
x descarta <ABRE_CHAVE,'{'> ... <FECHA_CHAVE,'}'>
v consome <FECHA_CHAVE,'}'>   (7:1)
```

A rejeição se explica pela gramática. Na regra `se = "se"` `" (" expr )" bloco`, o parser consome `se` e `(` e chama `expr`, que reconhece `x > 10` e não falso por inteiro. Ao retornar, o próximo token é `ABRE_CHAVE`. Nenhuma regra de expressão aceita esse token como continuação: { não é operador, e por isso `expr` termina. A produção de `se`, por sua vez, exige `FECHA_PAR` nessa posição. A chamada `esperar(FECHA_PAR)` falha, e o erro é emitido na coluna do {. A recuperação descarta o bloco {...} com as chaves balanceadas, e o } da linha 7 fecha corretamente o bloco principal. Desse modo, apenas um erro é relatado, sem mensagens em cascata.

B. Testes automatizados

Por fim, 28 testes automatizados (`unittest`) cobrem a distinção entre palavras reservadas e identificadores, a prioridade entre as expressões regulares, a contagem de linhas e colunas, a precedência e a associatividade dos operadores, todos os comandos da linguagem, as mensagens de erro e o número de erros relatados após a recuperação. Todos os testes passam.

VII. CONCLUSÃO

Este trabalho apresentou a especificação da linguagem Maré e a implementação dos seus analisadores léxico e sintático. Descrever os tokens por expressões regulares ordenadas resolveu as ambiguidades léxicas de forma simples e verificável. Uma gramática LL projetada com cuidado, por sua vez, permitiu um parser descendente recursivo curto e legível, em que a precedência dos operadores é consequência direta da estrutura das regras. A opção por uma implementação sem geradores exigiu mais código, mas deu à equipe controle total sobre as mensagens de erro e sobre a recuperação em modo pânico.

Nas próximas etapas, a AST será a entrada da análise semântica, com tabela de símbolos, verificação de escopo e checagem de tipos (por exemplo, rejeitar `seja x: inteiro = "a";`). Em seguida, será construído um interpretador ou gerador de código, o que completará o protótipo de compilador da Maré.

O código-fonte está disponível em <https://github.com/gustavoteixeirabittencourt/mare>.

REFERÊNCIAS

- [1] A. V. Aho, M. S. Lam, R. Sethi e J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2. ed. Boston, MA, EUA: Addison-Wesley, 2006.
- [2] K. D. Cooper e L. Torczon, *Engineering a Compiler*, 2. ed. Burlington, MA, EUA: Morgan Kaufmann, 2011.
- [3] J. E. Hopcroft, R. Motwani e J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3. ed. Boston, MA, EUA: Addison-Wesley, 2006.
- [4] K. C. Loudon, *Compiladores: Princípios e Práticas*. São Paulo, Brasil: Thomson Learning, 2004.
- [5] D. Grune e C. J. H. Jacobs, *Parsing Techniques: A Practical Guide*, 2. ed. Nova York, NY, EUA: Springer, 2008.
- [6] N. Wirth, "What can we do about the unnecessary diversity of notation for syntactic definitions?" *Communications of the ACM*, vol. 20, n. 11, pp. 822–823, 1977.
- [7] R. Nystrom, *Crafting Interpreters*. Genvener Benning, 2021.
- [8] Python Software Foundation, "re — Regular expression operations," *Python 3 Documentation*. [Online]. Disponível em: <https://docs.python.org/3/library/re.html>